Frontend Specification Document

Planetic Web Automated Website, Hosting, Domain, DNS, Billing & Client Portal Platform

---

1. Document Purpose

This document defines the complete frontend and integration specification for the Planetic Web platform.

The platform will allow customers to:

- Buy a complete bespoke website package
- Search and register domains
- Buy hosting packages
- Pay online
- Create and manage their account
- View domains, hosting, invoices, renewals, and support tickets
- Complete website project intake forms

The platform will connect with:

- Stripe for payments
- NameSilo API for domain registration
- Namecheap Domain API as backup registrar
- Namecheap WHM API for hosting automation
- Cloudflare API for DNS and security
- Namecheap cPanel Email SMTP with Jellyfish filtering for email delivery

The frontend should be clean, trustworthy, fast, accessible, mobile-friendly, and suitable for non-technical business customers.

---

2. Design Direction

2.1 Brand Personality

The design should feel:

- Professional
- Reliable
- Secure
- Modern
- Hosting-focused
- Business-friendly
- Simple for non-technical users

The customer should feel that Planetic Web can handle the technical work for them.

2.2 Visual Style

Use a modern SaaS/hosting style:

- Dark navy hero sections
- Bright blue and cyan accents
- White cards
- Soft shadows
- Rounded corners
- Clear pricing cards
- Dashboard-style service panels
- Strong call-to-action buttons
- Trust indicators such as SSL, Cloudflare, cPanel, secure billing, and support

2.3 Accessibility Target

The platform should target:

```
WCAG 2.2 Level AA
```

This should be treated as the accessibility standard for ADA-friendly implementation.

Important accessibility requirements:

- Normal text contrast should be at least 4.5:1.
- Large text contrast should be at least 3:1.
- Interactive controls should have visible focus states.
- All forms need labels.
- All buttons need clear text.
- Do not rely on colour alone to show status.
- All pages must be keyboard usable.
- Modals must trap focus.
- Error messages must be readable by screen readers.

---

3. Colour System

3.1 Brand Colour Palette

The palette below is designed to match the current Planetic Solution digital hosting direction: navy base, strong blue action colour, cyan technology accent, green success indicators, and clean white surfaces.

Primary Colours

Token	Hex	Usage
`--color-primary-950`	`#07111F`	Main dark hero background, footer, dashboard sidebar
`--color-primary-900`	`#0B1B33`	Dark sections, header background
`--color-primary-800`	`#102A4C`	Dark cards, secondary dark surfaces
`--color-primary-700`	`#163B68`	Hover states on dark backgrounds
`--color-primary-600`	`#1D4ED8`	Primary brand blue
`--color-primary-500`	`#2563EB`	Primary buttons and links
`--color-primary-400`	`#3B82F6`	Button hover, highlights
`--color-primary-100`	`#DBEAFE`	Light blue background badges
`--color-primary-50`	`#EFF6FF`	Very light blue sections

Accent Colours

Token	Hex	Usage
`--color-accent-cyan`	`#06B6D4`	Cloudflare/DNS highlights, icons, accent lines
`--color-accent-sky`	`#0EA5E9`	Secondary CTA, dashboard highlights
`--color-accent-indigo`	`#4F46E5`	Premium/Pro package accents
`--color-accent-violet`	`#7C3AED`	Agency/Ecommerce package accent

Neutral Colours

Token	Hex	Usage
`--color-white`	`#FFFFFF`	Main page background, card background
`--color-slate-50`	`#F8FAFC`	Light section background
`--color-slate-100`	`#F1F5F9`	Soft card backgrounds
`--color-slate-200`	`#E2E8F0`	Borders and dividers
`--color-slate-300`	`#CBD5E1`	Disabled borders
`--color-slate-500`	`#64748B`	Muted text
`--color-slate-600`	`#475569`	Secondary text
`--color-slate-700`	`#334155`	Strong secondary text
`--color-slate-900`	`#0F172A`	Main text colour

Status Colours

Token	Hex	Usage
`--color-success`	`#16A34A`	Active, paid, completed
`--color-success-bg`	`#DCFCE7`	Success badge background
`--color-warning`	`#F59E0B`	Pending, renewal soon
`--color-warning-bg`	`#FEF3C7`	Warning badge background
`--color-danger`	`#DC2626`	Failed, overdue, suspended
`--color-danger-bg`	`#FEE2E2`	Error badge background
`--color-info`	`#0284C7`	Information notices
`--color-info-bg`	`#E0F2FE`	Info badge background

---

4. Accessibility Colour Rules

4.1 Text Colour Rules

Use these combinations:

Background	Text Colour	Usage
`#FFFFFF`	`#0F172A`	Main content
`#F8FAFC`	`#0F172A`	Light sections
`#07111F`	`#FFFFFF`	Dark hero/footer
`#0B1B33`	`#FFFFFF`	Dark cards
`#2563EB`	`#FFFFFF`	Primary button
`#16A34A`	`#FFFFFF`	Success button
`#DC2626`	`#FFFFFF`	Danger button

Avoid light grey text on white backgrounds. Do not use `#94A3B8` for important body text.

4.2 Button Contrast Rules

Primary buttons must use:

```
background: #2563EB;
color: #FFFFFF;
```

Primary hover:

```
background: #1D4ED8;
color: #FFFFFF;
```

Danger buttons:

```
background: #DC2626;
color: #FFFFFF;
```

Success buttons:

```
background: #16A34A;
color: #FFFFFF;
```

4.3 Focus Ring

All interactive elements must have a visible focus ring:

```
outline: 3px solid #06B6D4;
outline-offset: 3px;
```

Do not remove focus outlines unless replacing them with an equally visible focus style.

---

5. Typography System

5.1 Font Family

Recommended primary font:

```
Inter
```

Fallback stack:

```
font-family: Inter, ui-sans-serif, system-ui, -apple-system,
BlinkMacSystemFont, "Segoe UI", sans-serif;
```

5.2 Optional Heading Font

Optional heading font:

```
Space Grotesk
```

Use only if you want a more modern tech feel.

Fallback:

```
font-family: "Space Grotesk", Inter, ui-sans-serif, system-ui, sans-serif;
```

5.3 Font Sizes

Token	Size	Line Height	Usage
`text-xs`	12px	16px	Small labels, helper text
`text-sm`	14px	20px	Secondary text
`text-base`	16px	24px	Body text
`text-lg`	18px	28px	Lead text
`text-xl`	20px	30px	Card headings
`text-2xl`	24px	32px	Section headings
`text-3xl`	30px	38px	Page headings
`text-4xl`	36px	44px	Hero heading mobile
`text-5xl`	48px	56px	Hero heading desktop
`text-6xl`	60px	68px	Large marketing hero only

5.4 Font Weights

Weight	Usage
400	Body text
500	Labels, nav items
600	Buttons, card titles
700	Section headings
800	Hero headings

5.5 Typography Rules

- Body text should be at least 16px.
- Do not use text smaller than 14px for important information.
- Form labels should be 14px or 16px.
- Line height should be generous for readability.

- Avoid long paragraphs in dashboard pages.
- Use bullet lists for package features.
- Use real text instead of image text.

---

6. Spacing System

Use an 8px spacing scale.

Token	Pixels	Usage
`space-1`	4px	Tiny gaps
`space-2`	8px	Icon/text gap
`space-3`	12px	Small component padding
`space-4`	16px	Default spacing
`space-5`	20px	Card internal spacing
`space-6`	24px	Form/group spacing
`space-8`	32px	Section component spacing
`space-10`	40px	Page section spacing
`space-12`	48px	Large section spacing
`space-16`	64px	Hero/major sections
`space-20`	80px	Desktop section spacing
`space-24`	96px	Large hero spacing

Layout Rules

- Mobile page padding: 16px
- Tablet page padding: 24px
- Desktop page padding: 32px
- Maximum content width: 1200px
- Maximum dashboard content width: 1440px
- Card gap: 24px
- Pricing card gap: 24px
- Form field gap: 16px
- Section gap: 64px desktop, 40px mobile

---

7. Border Radius

Token	Radius	Usage
`radius-sm`	6px	Small tags, badges
`radius-md`	10px	Inputs, small buttons
`radius-lg`	14px	Cards
`radius-xl`	18px	Pricing cards, feature cards
`radius-2xl`	24px	Hero panels, modals
`radius-full`	999px	Pills and badges

Recommended default:

```
border-radius: 14px;
```

---

8. Shadow System

Use soft shadows, not heavy shadows.

```
--shadow-sm: 0 1px 2px rgba(15, 23, 42, 0.06);
--shadow-md: 0 8px 24px rgba(15, 23, 42, 0.08);
--shadow-lg: 0 18px 50px rgba(15, 23, 42, 0.12);
--shadow-dark: 0 18px 50px rgba(0, 0, 0, 0.28);
```

Usage:

- Cards: `shadow-sm` or `shadow-md`
- Modals: `shadow-lg`
- Dark hero console card: `shadow-dark`
- Buttons: no heavy shadow unless CTA section

---

9. Layout System

9.1 Public Website Layout

Main pages:

- Home
- Domains

- Hosting
- Website Package
- Checkout
- Contact
- Legal pages

9.2 Public Header

Desktop header:

- Logo left
- Navigation center/left
- Client Login button
- Get Started button

Mobile header:

- Logo left
- Menu button right
- Slide-down or drawer menu
- Client Login and Get Started inside mobile menu

Header height:

```
72px desktop
64px mobile
```

Header style:

```
background: #FFFFFF;
border-bottom: 1px solid #E2E8F0;
```

On dark hero pages, header can stay white for clarity.

9.3 Hero Section

Hero should use:

```
background: linear-gradient(135deg, #07111F 0%, #0B1B33 55%, #102A4C 100%);
color: #FFFFFF;
```

Hero content should include:

- Eyebrow text: “Premium Hosting, Domains & Websites”
- Main headline
- Short paragraph
- Domain search bar

- CTA buttons
- Trust badges
- Cloud console preview card

9.4 Dashboard Layout

Dashboard should use:

- Left sidebar on desktop
- Top mobile navigation on small screens
- Main content area
- Cards and status badges
- Clear page titles

Desktop layout:

```
Sidebar: 280px
Main content: flexible
Top bar: 72px
```

Mobile layout:

```
Top bar
Collapsible menu
Single-column cards
```

---

10. Component System

10.1 Buttons

Primary Button

Usage:

- Get Started
- Search Domain
- Pay Now
- Create Account
- Save Changes

Style:

```
background: #2563EB;
color: #FFFFFF;
border: 1px solid #2563EB;
```

```
border-radius: 10px;
padding: 12px 20px;
font-size: 16px;
font-weight: 600;
min-height: 44px;
```

Hover:

```
background: #1D4ED8;
border-color: #1D4ED8;
```

Focus:

```
outline: 3px solid #06B6D4;
outline-offset: 3px;
```

Disabled:

```
background: #CBD5E1;
border-color: #CBD5E1;
color: #64748B;
cursor: not-allowed;
```

Secondary Button

Usage:

- View Hosting Plans
- Learn More
- Back to Dashboard

Style:

```
background: #FFFFFF;
color: #0F172A;
border: 1px solid #CBD5E1;
border-radius: 10px;
padding: 12px 20px;
font-weight: 600;
min-height: 44px;
```

Hover:

```
background: #F8FAFC;
border-color: #94A3B8;
```

Dark Hero Secondary Button

Use on dark backgrounds:

```
background: rgba(255,255,255,0.08);
color: #FFFFFF;
border: 1px solid rgba(255,255,255,0.24);
```

Danger Button

Usage:

- Cancel service
- Delete draft
- Confirm suspension

Style:

```
background: #DC2626;
color: #FFFFFF;
border: 1px solid #DC2626;
```

Button Accessibility Rules

- Minimum touch target: 44px height.
- Button text must be descriptive.
- Avoid “Click here”.
- Loading state must show spinner and text.
- Disabled buttons must explain why when needed.

Example loading text:

```
Processing payment...
Creating hosting...
Checking domain...
```

10.2 Inputs

Text Input

Style:

```
background: #FFFFFF;
color: #0F172A;
border: 1px solid #CBD5E1;
border-radius: 10px;
padding: 12px 14px;
font-size: 16px;
min-height: 44px;
```

Focus:

```
border-color: #2563EB;
box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.15);
outline: none;
```

Error:

```
border-color: #DC2626;
background: #FFF7F7;
```

Success:

```
border-color: #16A34A;
```

Input Rules

Every input must have:

- Visible label
- Helper text if needed
- Error message area
- Proper autocomplete attributes
- Keyboard support
- Screen-reader accessible validation

Example:

```
Label: Domain name
Placeholder: example.com
Helper: Search for your business domain.
Error: Please enter a valid domain name.
```

10.3 Domain Search Bar

The domain search component is one of the most important conversion elements.

Desktop layout:

```
[ Search domain name input                ] [Search Domain button]
```

Mobile layout:

```
[ Search domain name input ]
[ Search Domain button     ]
```

States:

1. Empty
2. Typing
3. Checking
4. Available
5. Unavailable
6. Premium domain
7. API error

Available result card:

```
example.com is available
£12.99/year
[Add to Cart]
```

Unavailable result card:

```
example.com is not available
Try one of these alternatives:
example.co.uk
example.net
example.org
```

Accessibility:

- Announce result with `aria-live="polite"`.
- Do not show result by colour alone.
- Use text and icon.

10.4 Cards

Standard Card

Usage:

- Feature cards
- Dashboard service cards
- Support cards

Style:

```
background: #FFFFFF;
border: 1px solid #E2E8F0;
border-radius: 18px;
padding: 24px;
box-shadow: 0 8px 24px rgba(15, 23, 42, 0.08);
```

Dashboard Card

Style:

```
background: #FFFFFF;
border: 1px solid #E2E8F0;
border-radius: 16px;
padding: 20px;
```

Dark Console Card

Usage:

- Hero “Planetic Cloud Console” preview

Style:

```
background: rgba(15, 23, 42, 0.86);
border: 1px solid rgba(255,255,255,0.14);
border-radius: 24px;
box-shadow: 0 18px 50px rgba(0,0,0,0.28);
color: #FFFFFF;
```

10.5 Pricing Cards

Pricing cards should be clear and easy to compare.

Each pricing card includes:

- Plan name
- Short description
- Monthly price
- Yearly price
- Storage
- Bandwidth
- Email accounts
- Feature list
- CTA button
- Optional badge such as “Recommended”

Recommended plan style:

```
border: 2px solid #2563EB;
box-shadow: 0 18px 50px rgba(37, 99, 235, 0.18);
```

Badge:

```
background: #DBEAFE;
color: #1D4ED8;
border-radius: 999px;
padding: 6px 10px;
font-size: 12px;
font-weight: 700;
```

10.6 Status Badges

Badges must use both colour and text.

Status	Text	Background	Text Colour
Active	Active	`#DCFCE7`	`#166534`
Paid	Paid	`#DCFCE7`	`#166534`
Pending	Pending	`#FEF3C7`	`#92400E`
Failed	Failed	`#FEE2E2`	`#991B1B`
Suspended	Suspended	`#FEE2E2`	`#991B1B`
Manual Review	Manual Review	`#E0F2FE`	`#075985`
Provisioning	Provisioning	`#DBEAFE`	`#1D4ED8`

10.7 Modals

Use modals for:

- Confirm cancellation
- Confirm DNS change
- Confirm hosting suspension
- Payment method update redirect
- Delete uploaded file confirmation

Modal style:

```
background: #FFFFFF;
border-radius: 24px;
box-shadow: 0 18px 50px rgba(15, 23, 42, 0.18);
max-width: 560px;
padding: 24px;
```

Overlay:

```
background: rgba(7, 17, 31, 0.72);
```

Accessibility rules:

- Modal must trap keyboard focus.
- Escape key closes modal unless action is critical.
- First focus should move to modal heading or first field.
- Modal must have `aria-modal="true"`.
- Modal must have labelled heading.
- Focus must return to original button when closed.

10.8 Tables

Use tables for:

- Invoices
- Domains
- Hosting accounts
- DNS records
- Support tickets

Table design:

```
background: #FFFFFF;
border: 1px solid #E2E8F0;
```

```
border-radius: 16px;
overflow: hidden;
```

Header:

```
background: #F8FAFC;
color: #334155;
font-size: 14px;
font-weight: 600;
```

Rows:

```
border-bottom: 1px solid #E2E8F0;
```

Mobile:

- Convert tables into stacked cards.
- Do not force horizontal scrolling unless needed for DNS records.

---

11. Page Specifications

11.1 Home Page

Sections:

1. Header
2. Hero with domain search
3. Trust badges
4. Hosting/services overview
5. Hosting plans
6. Website package offer
7. Why choose us
8. Testimonials
9. FAQs
10. Final CTA
11. Footer

Hero CTA buttons:

```
Search Domain
View Hosting Plans
Get Website for £200
```

11.2 Domain Search Page

Main features:

- Search input
- TLD price badges
- Availability result
- Suggested alternatives
- Add to cart
- Explanation of domain renewal

11.3 Hosting Plans Page

Show:

- Starter Hosting
- Business Hosting
- Pro Hosting
- Agency / Ecommerce Hosting

Each plan must clearly show:

- Monthly price
- Yearly price
- Storage
- Bandwidth
- Email accounts
- cPanel included
- SSL included
- Cloudflare setup support

11.4 Website Package Page

Main offer:

```
Complete Bespoke Website for £200
```

Must clearly say:

```
Free domain and hosting for the first year. Renewal applies after the first
year.
```

Sections:

- Hero offer
- What is included
- How it works
- Timeline

- FAQs
- Order CTA
- Renewal explanation

11.5 Checkout Page

Checkout must show:

- Selected package
- Selected domain
- Hosting plan
- First-year included items
- Renewal notice
- Total due today
- Secure payment message
- Login/register section
- Stripe checkout button

Important:

The frontend must not calculate final trusted prices. It can display estimates, but backend must calculate the actual price.

11.6 Customer Dashboard

Dashboard home cards:

- Active domains
- Active hosting
- Open invoices
- Renewal reminders
- Website project status
- Support tickets

Navigation:

```
Dashboard
Domains
Hosting
Billing
Website Projects
Support
Account Settings
Logout
```

11.7 Admin Dashboard

Admin should use Filament or a similar admin UI.

Main admin sections:

- Customers
- Orders
- Payments
- Invoices
- Domains
- Hosting Accounts
- Cloudflare Zones
- DNS Records
- Website Projects
- Support Tickets
- Provisioning Logs
- Settings

---

12. Responsive Breakpoints

Use these breakpoints:

```
sm: 640px
md: 768px
lg: 1024px
xl: 1280px
2xl: 1536px
```

Mobile Rules

- Single column layout
- Full-width buttons
- Sticky bottom CTA only on important landing pages
- Pricing cards stacked
- Tables converted to cards
- Header becomes hamburger menu

Tablet Rules

- Two-column cards where possible
- Domain search remains horizontal if space allows

Desktop Rules

- Max width 1200px for marketing pages
- Max width 1440px for dashboard
- Multi-column pricing cards
- Sidebar dashboard layout

---

13. ADA / WCAG Accessibility Requirements

13.1 Keyboard Access

Users must be able to:

- Navigate all menus by keyboard
- Open and close modals
- Submit forms
- Use domain search
- Complete checkout
- Open support tickets
- Navigate dashboard

13.2 Skip Link

Every page should include a skip link:

```
Skip to main content
```

It should appear when focused.

13.3 Forms

Every form field must have:

- Label
- Error message
- Helper text if needed
- Proper autocomplete
- Clear required field indicator
- Keyboard focus state

13.4 Images

Every meaningful image must have alt text.

Decorative images should use empty alt:

```
alt=""
```

13.5 Live Status Updates

Use `aria-live="polite"` for:

- Domain search results

- Checkout status
- Provisioning status
- Form submission result

13.6 Error Messages

Error messages should be:

- Visible
- Specific
- Next to the relevant field
- Screen-reader accessible
- Not colour-only

Example:

```
Please enter a valid email address.
```

Do not use only:

```
This field is wrong.
```

---

14. Frontend API Rule

The frontend must never call third-party services directly.

The frontend calls only Planetic Web Laravel backend routes.

Correct flow:

```
Frontend → Laravel backend → Third-party API
```

Incorrect flow:

```
Frontend → Stripe secret API
Frontend → Cloudflare API
Frontend → WHM API
Frontend → NameSilo API
```

Reason:

Third-party API keys must remain private and must never be exposed in browser JavaScript.

---

15. Internal Frontend-to-Backend API Specification

15.1 Domain Search

Endpoint:

```
POST /api/domains/search
```

Request:

```
{
  "domain": "example.com"
}
```

Response — available:

```
{
  "success": true,
  "domain": "example.com",
  "available": true,
  "price": "12.99",
  "currency": "GBP",
  "premium": false,
  "suggestions": [
    {
      "domain": "example.co.uk",
      "available": true,
      "price": "9.99"
    }
  ]
}
```

Response — unavailable:

```
{
  "success": true,
  "domain": "example.com",
  "available": false,
  "suggestions": [
    {
      "domain": "example.co.uk",
      "available": true,
      "price": "9.99"
```

```
    }
  ]
}
```

Frontend states:

- Loading
- Available
- Unavailable
- Error

15.2 Add Item to Cart

Endpoint:

```
POST /api/cart/items
```

Request:

```
{
  "item_type": "website_package",
  "product_id": 1,
  "domain_name": "example.com",
  "billing_cycle": "one_time"
}
```

Response:

```
{
  "success": true,
  "cart": {
    "id": 15,
    "subtotal": "200.00",
    "total": "200.00",
    "currency": "GBP",
    "items": [
      {
        "name": "Complete Bespoke Website",
        "domain_name": "example.com",
        "total": "200.00"
      }
    ]
  }
}
```

15.3 Start Checkout

Endpoint:

```
POST /api/checkout/start
```

Request:

```
{
  "cart_id": 15,
  "customer_details": {
    "name": "John Smith",
    "email": "john@example.com",
    "phone": "+447000000000",
    "company_name": "Example Ltd"
  }
}
```

Response:

```
{
  "success": true,
  "checkout_url": "https://checkout.stripe.com/c/pay/...",
  "stripe_checkout_session_id": "cs_live_..."
}
```

Frontend action:

Redirect user to `checkout_url`.

15.4 Customer Dashboard Summary

Endpoint:

```
GET /api/dashboard/summary
```

Response:

```
{
  "domains_count": 2,
  "hosting_accounts_count": 1,
  "open_invoices_count": 0,
  "next_renewal_date": "2027-06-01",
```

```
  "website_project": {
    "status": "design_in_progress"
  }
}
```

15.5 Website Project Intake

Endpoint:

```
POST /api/website-projects/{project_id}/intake
```

Request:

```
{
  "business_name": "Example Ltd",
  "business_description": "A local cleaning business.",
  "industry": "Cleaning Services",
  "pages_required": ["Home", "About", "Services", "Contact"],
  "brand_colours": "Blue and white",
  "reference_websites": ["https://example-reference.com"],
  "special_requirements": "Need WhatsApp button and contact form."
}
```

Response:

```
{
  "success": true,
  "message": "Website project details saved successfully.",
  "status": "content_received"
}
```

15.6 Support Ticket Create

Endpoint:

```
POST /api/support/tickets
```

Request:

```
{
  "subject": "I need help with my domain",
```

```
  "category": "domain",
  "message": "My domain is not loading yet."
}
```

Response:

```
{
  "success": true,
  "ticket_number": "TCK-10045",
  "status": "open"
}
```

---

16. Third-Party Integration Specification

All third-party integrations must run on the backend only.

---

17. Stripe Integration

17.1 Purpose

Stripe handles:

- One-time payment for website package
- Hosting subscriptions
- Domain renewal payments
- Invoice events
- Failed payment notifications
- Payment method management

17.2 Create Checkout Session

Backend calls:

```
POST https://api.stripe.com/v1/checkout/sessions
```

Used when:

- Customer clicks Pay Now
- Customer purchases website package
- Customer buys hosting
- Customer buys domain

Data sent:

```
{
  "mode": "payment",
  "customer_email": "john@example.com",
  "line_items": [
    {
      "price_data": {
        "currency": "gbp",
        "product_data": {
          "name": "Complete Bespoke Website"
        },
        "unit_amount": 20000
      },
      "quantity": 1
    }
  ],
  "success_url": "https://planeticweb.com/checkout/success?
session_id={CHECKOUT_SESSION_ID}",
  "cancel_url": "https://planeticweb.com/checkout/cancel",
  "metadata": {
    "order_id": "ORD-10001",
    "domain_name": "example.com",
    "package_type": "website_package"
  }
}
```

Expected response:

```
{
  "id": "cs_live_xxx",
  "object": "checkout.session",
  "url": "https://checkout.stripe.com/c/pay/..."
}
```

Frontend action:

Redirect customer to the returned Stripe checkout URL.

Security rule:

Do not create domain or hosting yet. Wait for verified webhook.

17.3 Stripe Webhook

Stripe calls:

```
POST https://planeticweb.com/webhooks/stripe
```

Events to handle:

```
checkout.session.completed
payment_intent.succeeded
payment_intent.payment_failed
invoice.paid
invoice.payment_failed
customer.subscription.created
customer.subscription.updated
customer.subscription.deleted
```

Expected webhook data:

```
{
  "id": "evt_xxx",
  "type": "checkout.session.completed",
  "data": {
    "object": {
      "id": "cs_live_xxx",
      "payment_status": "paid",
      "metadata": {
        "order_id": "ORD-10001"
      }
    }
  }
}
```

Backend action:

- Verify Stripe signature
- Check if event already processed
- Mark order as paid
- Start provisioning queue
- Send confirmation email

Failure handling:

- Invalid signature: reject
- Duplicate event: ignore safely
- Missing order: log and mark manual review
- Payment unpaid: do not provision

---

18. NameSilo API Integration

18.1 Purpose

NameSilo is the preferred registrar for:

- Domain availability
- Domain registration
- Domain renewal
- Domain information
- Nameserver update
- WHOIS privacy where available

18.2 API Base Format

NameSilo API uses GET requests:

```
GET https://www.namesilo.com/api/{operation}?
version=1&type=json&key={API_KEY}
```

18.3 Check Domain Availability

Backend call:

```
GET https://www.namesilo.com/api/checkRegisterAvailability
```

Data sent as query parameters:

```
version=1
type=json
key=NAMESILO_API_KEY
domains=example.com
```

Expected response:

```
{
  "reply": {
    "code": 300,
    "detail": "success",
    "available": {
      "domain": "example.com",
      "price": "12.99"
    }
```

```
  }
}
```

Backend returns simplified response to frontend:

```
{
  "available": true,
  "domain": "example.com",
  "price": "12.99"
}
```

18.4 Register Domain

Backend call:

```
GET https://www.namesilo.com/api/registerDomain
```

Data sent:

```
version=1
type=json
key=NAMESILO_API_KEY
domain=example.com
years=1
private=1
auto_renew=1
contact_id=CONTACT_ID
```

Expected response:

```
{
  "reply": {
    "code": 300,
    "detail": "success",
    "domain": "example.com",
    "order_amount": "12.99"
  }
}
```

Backend action:

- Store domain as active
- Store registration date
- Store expiry date if available
- Create Cloudflare zone

• Update nameservers after Cloudflare returns nameservers

18.5 Update Nameservers

Backend call:

```
GET https://www.namesilo.com/api/changeNameServers
```

Data sent:

```
version=1
type=json
key=NAMESILO_API_KEY
domain=example.com
ns1=cloudflare-ns1
ns2=cloudflare-ns2
```

Expected response:

```
{
  "reply": {
    "code": 300,
    "detail": "success"
  }
}
```

18.6 Get Domain Info

Backend call:

```
GET https://www.namesilo.com/api/getDomainInfo
```

Data sent:

```
version=1
type=json
key=NAMESILO_API_KEY
domain=example.com
```

Expected response:

```
{
  "reply": {
    "code": 300,
```

```
    "domain": "example.com",
    "expires": "2027-06-01",
    "status": "Active"
  }
}
```

18.7 Renew Domain

Backend call:

```
GET https://www.namesilo.com/api/renewDomain
```

Data sent:

```
version=1
type=json
key=NAMESILO_API_KEY
domain=example.com
years=1
```

Expected response:

```
{
  "reply": {
    "code": 300,
    "detail": "success",
    "domain": "example.com",
    "order_amount": "12.99"
  }
}
```

Important rule:

Only renew domain after customer payment is confirmed.

---

19. Namecheap Domain API Integration

19.1 Purpose

Namecheap Domain API is a backup registrar option.

It can be used for:

- Domain registration

- Domain renewal
- Domain nameserver updates
- Domain info
- DNS host records if using Namecheap DNS

For this app, Cloudflare is the DNS provider, so Namecheap DNS records are not the main method. The most important Namecheap operation is setting custom nameservers to Cloudflare.

19.2 API Base URL

```
GET https://api.namecheap.com/xml.response
```

Required common parameters:

```
ApiUser
ApiKey
UserName
Command
ClientIp
```

19.3 Set Custom Nameservers

Backend call:

```
GET https://api.namecheap.com/xml.response
```

Data sent:

```
ApiUser=NAMECHEAP_API_USER
ApiKey=NAMECHEAP_API_KEY
UserName=NAMECHEAP_USERNAME
Command=namecheap.domains.dns.setCustom
ClientIp=SERVER_IP
SLD=example
TLD=com
NameServers=ns1.cloudflare.com,ns2.cloudflare.com
```

Expected response:

```
<ApiResponse Status="OK">
  <CommandResponse Type="namecheap.domains.dns.setCustom">
    <DomainDNSSetCustomResult Domain="example.com" Updated="true" />
  </CommandResponse>
</ApiResponse>
```

Failure handling:

- If API returns error, mark nameserver update as failed.
- Notify Technical Admin.
- Do not retry endlessly.
- Customer sees “DNS activation pending”.

---

20. Cloudflare API Integration

20.1 Purpose

Cloudflare manages:

- DNS zones
- DNS records
- Nameserver assignment
- SSL/TLS setting
- Always Use HTTPS
- CDN/proxy for website records

20.2 Create Zone

Backend call:

```
POST https://api.cloudflare.com/client/v4/zones
```

Headers:

```
Authorization: Bearer CLOUDFLARE_API_TOKEN
Content-Type: application/json
```

Data sent:

```
{
  "name": "example.com",
  "account": {
    "id": "CLOUDFLARE_ACCOUNT_ID"
  },
  "jump_start": false,
  "type": "full"
}
```

Expected response:

```
{
  "success": true,
  "result": {
    "id": "zone_id_here",
    "name": "example.com",
    "status": "pending",
    "name_servers": [
      "ns1.cloudflare.com",
      "ns2.cloudflare.com"
    ]
  }
}
```

Backend action:

- Store Cloudflare zone ID
- Store assigned nameservers
- Send nameservers to registrar
- Start DNS record creation job

20.3 Create DNS Record

Backend call:

```
POST https://api.cloudflare.com/client/v4/zones/{zone_id}/dns_records
```

Headers:

```
Authorization: Bearer CLOUDFLARE_API_TOKEN
Content-Type: application/json
```

Example A record:

```
{
  "type": "A",
  "name": "@",
  "content": "203.0.113.10",
  "ttl": 1,
  "proxied": true
}
```

Expected response:

```
{
  "success": true,
  "result": {
    "id": "dns_record_id",
    "type": "A",
    "name": "example.com",
    "content": "203.0.113.10",
    "proxied": true
  }
}
```

Default records to create:

```
A       @          WHM_SERVER_IP          proxied
CNAME   www        example.com            proxied
A       mail       WHM_SERVER_IP          DNS only
A       cpanel     WHM_SERVER_IP          DNS only
A       webmail    WHM_SERVER_IP          DNS only
MX      @          mail.example.com       DNS only
TXT     @          SPF record             DNS only
TXT     _dmarc     DMARC record           DNS only
```

Important:

Do not proxy mail, cPanel, webmail, WHM, FTP, MX, SPF, DKIM, or DMARC records.

20.4 Update DNS Record

Backend call:

```
PATCH https://api.cloudflare.com/client/v4/zones/{zone_id}/dns_records/{record_id}
```

Data sent:

```
{
  "content": "203.0.113.20",
  "proxied": true
}
```

Expected response:

```
{
  "success": true,
```

```
  "result": {
    "id": "dns_record_id",
    "content": "203.0.113.20"
  }
}
```

20.5 Delete DNS Record

Backend call:

```
DELETE https://api.cloudflare.com/client/v4/zones/{zone_id}/dns_records/{record_id}
```

Expected response:

```
{
  "success": true,
  "result": {
    "id": "dns_record_id"
  }
}
```

For MVP, DNS editing can be admin-only. Customer DNS editing can be added later.

20.6 Set SSL Mode

Backend call:

```
PATCH https://api.cloudflare.com/client/v4/zones/{zone_id}/settings/ssl
```

Data sent:

```
{
  "value": "full"
}
```

Expected response:

```
{
  "success": true,
  "result": {
    "id": "ssl",
```

```
    "value": "full"
  }
}
```

Recommended value:

```
full
```

Later, use `full_strict` when origin certificates are correctly installed.

20.7 Enable Always Use HTTPS

Backend call:

```
PATCH https://api.cloudflare.com/client/v4/zones/{zone_id}/settings/always_use_https
```

Data sent:

```
{
  "value": "on"
}
```

Expected response:

```
{
  "success": true,
  "result": {
    "id": "always_use_https",
    "value": "on"
  }
}
```

---

21. Namecheap WHM / cPanel API Integration

21.1 Purpose

WHM API is used to create and manage cPanel hosting accounts on Namecheap reseller hosting.

21.2 Base URL

```
https://WHM_HOST:2087/json-api/{function}?api.version=1
```

Authentication header:

```
Authorization: whm WHM_USERNAME:WHM_API_TOKEN
```

All WHM calls must happen from the backend only.

21.3 Create cPanel Account

Backend call:

```
GET https://WHM_HOST:2087/json-api/createacct?api.version=1
```

Data sent as query parameters:

```
username=exampleusr
domain=example.com
contactemail=john@example.com
plan=planetic_starter
password=SECURE_GENERATED_PASSWORD
dkim=1
spf=1
dmarc=1
cgi=1
```

Expected response:

```
{
  "metadata": {
    "result": 1,
    "reason": "Account Creation Ok",
    "command": "createacct"
  },
  "data": {
    "ip": "203.0.113.10",
    "nameserver": "ns1.example.com",
    "package": "planetic_starter"
  }
}
```

Backend action:

- Store WHM username
- Store server IP
- Store cPanel URL
- Store hosting status as active
- Trigger Cloudflare DNS records job

Failure response:

```
{
  "metadata": {
    "result": 0,
    "reason": "Account already exists"
  }
}
```

Failure handling:

- Mark hosting as failed/manual review
- Notify Technical Admin
- Do not retry blindly if duplicate risk exists

21.4 Suspend Hosting Account

Backend call:

```
GET https://WHM_HOST:2087/json-api/suspendacct?api.version=1
```

Data sent:

```
user=exampleusr
reason=Payment overdue
```

Expected response:

```
{
  "metadata": {
    "result": 1,
    "reason": "OK",
    "command": "suspendacct"
  }
}
```

Used when:

- Hosting renewal payment fails
- Grace period ends
- Admin manually suspends service

21.5 Unsuspend Hosting Account

Backend call:

```
GET https://WHM_HOST:2087/json-api/unsuspendacct?api.version=1
```

Data sent:

```
user=exampleusr
```

Expected response:

```
{
  "metadata": {
    "result": 1,
    "reason": "OK",
    "command": "unsuspendacct"
  }
}
```

Used when:

- Customer pays overdue invoice
- Admin manually reactivates service

21.6 Change Hosting Package

Backend call:

```
GET https://WHM_HOST:2087/json-api/changepackage?api.version=1
```

Data sent:

```
user=exampleusr
pkg=planetic_business
```

Expected response:

```
{
  "metadata": {
    "result": 1,
    "reason": "OK"
  }
}
```

Used when:

- Customer upgrades hosting
- Admin changes package manually

This can be nice-to-have after MVP.

21.7 List Accounts

Backend call:

```
GET https://WHM_HOST:2087/json-api/listaccts?api.version=1
```

Expected response:

```
{
  "acct": [
    {
      "user": "exampleusr",
      "domain": "example.com",
      "ip": "203.0.113.10",
      "plan": "planetic_starter",
      "suspended": 0
    }
  ]
}
```

Used for:

- Syncing hosting status
- Admin monitoring
- Detecting manual changes inside WHM

---

22. cPanel Email / Jellyfish Integration

22.1 Purpose

Namecheap cPanel Email will send transactional emails. Jellyfish will provide filtering/spam management.

Emails include:

- Account created
- Order confirmation
- Payment success
- Payment failed
- Domain registered
- Hosting account created
- Renewal reminder
- Hosting suspended
- Hosting reactivated
- Support ticket reply
- Website project update

22.2 SMTP Configuration

Backend Laravel mail settings:

```
MAIL_MAILER=smtp
MAIL_HOST=mail.planeticweb.com
MAIL_PORT=465
MAIL_USERNAME=no-reply@planeticweb.com
MAIL_PASSWORD=EMAIL_PASSWORD
MAIL_ENCRYPTION=ssl
MAIL_FROM_ADDRESS=no-reply@planeticweb.com
MAIL_FROM_NAME="Planetic Web"
```

Alternative:

```
MAIL_PORT=587
MAIL_ENCRYPTION=tls
```

22.3 Expected Email Send Result

Laravel should treat successful SMTP sending as:

```
{
  "success": true,
```

```
  "message": "Email sent"
}
```

If email fails:

```
{
  "success": false,
  "error": "SMTP authentication failed"
}
```

Backend action:

- Log email success/failure in `notification_logs`
- Allow admin to resend failed emails
- Do not block service provisioning if email sending fails

22.4 DNS Records for Email

Cloudflare must contain:

```
A      mail       cPanel server IP       DNS only
A      webmail    cPanel server IP       DNS only
MX     @          mail.planeticweb.com   DNS only
TXT    @          SPF record             DNS only
TXT    _dmarc     DMARC record           DNS only
TXT    DKIM       DKIM from cPanel       DNS only
```

Important:

Email records must not be proxied through Cloudflare.

---

23. Loading and Empty States

23.1 Loading States

Use clear loading text:

```
Checking domain availability...
Creating secure checkout...
Setting up your domain...
Creating your hosting account...
Loading invoices...
```

Do not show only a spinner.

23.2 Empty States

Examples:

No domains:

```
You do not have any domains yet.
Search and register your first domain to get started.
[Search Domain]
```

No invoices:

```
No invoices found.
Your invoices will appear here after your first purchase.
```

No support tickets:

```
You have no support tickets.
Need help? Create a support request.
[Create Ticket]
```

---

24. Error States

24.1 Customer Error Message Style

Use simple messages.

Example:

```
We could not complete this action right now. Please try again or contact
support.
```

Do not show raw API errors to customers.

Bad:

```
WHM API createacct returned metadata.result=0
```

Good:

```
Your hosting setup could not be completed automatically. Our team has been notified.
```

24.2 Error Components

Error alert style:

```
background: #FEE2E2;
border: 1px solid #FCA5A5;
color: #991B1B;
border-radius: 14px;
padding: 16px;
```

Warning alert:

```
background: #FEF3C7;
border: 1px solid #FCD34D;
color: #92400E;
```

Info alert:

```
background: #E0F2FE;
border: 1px solid #7DD3FC;
color: #075985;
```

Success alert:

```
background: #DCFCE7;
border: 1px solid #86EFAC;
color: #166534;
```

---

25. Frontend Security Rules

25.1 Never Trust Frontend Prices

Frontend may display prices, but backend must calculate final total.

Do not send trusted price from frontend like:

```
{
  "price": "200"
}
```

Instead send product ID and backend calculates price.

25.2 Never Expose API Keys

Never expose:

```
Stripe secret key
Stripe webhook secret
Cloudflare API token
WHM API token
NameSilo API key
Namecheap API key
SMTP password
```

25.3 Protect Hidden Fields

Users can edit hidden fields in browser tools. Never trust hidden fields for:

- Price
- User ID
- Order status
- Payment status
- Service status
- Admin permission
- Hosting package mapping

25.4 File Upload Rules

Allowed:

```
jpg
jpeg
png
webp
pdf
doc
docx
txt
```

Avoid unless necessary:

```
zip
```

Block:

```
php
exe
js
sh
bat
html
svg unless sanitized
```

---

26. Frontend Acceptance Criteria

26.1 Design

- Website uses Planetic-style navy/blue/cyan palette.
- All main pages are responsive.
- Buttons have visible focus states.
- Forms have labels.
- Cards and pricing sections are consistent.
- Dashboard is easy to understand for non-technical users.
- Status badges use text and colour.

26.2 Accessibility

- Keyboard users can navigate all pages.
- Colour contrast meets WCAG 2.2 AA.
- Inputs have labels and error messages.
- Modals trap focus.
- Domain search results are announced to screen readers.
- No important information is shown by colour only.
- Touch targets are at least 44px high.

26.3 Integration

- Frontend never calls third-party APIs directly.
- Domain search calls Laravel backend.
- Checkout calls Laravel backend.
- Stripe checkout URL is returned by backend.
- Provisioning starts only after verified Stripe webhook.
- Customer dashboard shows real backend status.
- Failed provisioning shows safe customer message.

---

27. Recommended Tailwind Theme Tokens

Add these colours to Tailwind config:

```
theme: {
  extend: {
    colors: {
      primary: {
        50: '#EFF6FF',
        100: '#DBEAFE',
        400: '#3B82F6',
        500: '#2563EB',
        600: '#1D4ED8',
        700: '#163B68',
        800: '#102A4C',
        900: '#0B1B33',
        950: '#07111F',
      },
      accent: {
        cyan: '#06B6D4',
        sky: '#0EA5E9',
        indigo: '#4F46E5',
        violet: '#7C3AED',
      },
      success: '#16A34A',
      warning: '#F59E0B',
      danger: '#DC2626',
      info: '#0284C7',
      slate: {
        50: '#F8FAFC',
        100: '#F1F5F9',
        200: '#E2E8F0',
        300: '#CBD5E1',
        500: '#64748B',
        600: '#475569',
        700: '#334155',
        900: '#0F172A',
      }
    },
    borderRadius: {
      xl: '18px',
      '2xl': '24px',
    },
    boxShadow: {
      soft: '0 8px 24px rgba(15, 23, 42, 0.08)',
      large: '0 18px 50px rgba(15, 23, 42, 0.12)',
      dark: '0 18px 50px rgba(0, 0, 0, 0.28)',
    }
```

```
  }
}
```

---

28. Final Frontend Recommendation

Build the frontend with:

```
Laravel Blade
Tailwind CSS
Alpine.js
Filament Admin Panel
Stripe Checkout redirect
Backend-only third-party API calls
WCAG 2.2 AA accessibility rules
```

The final product should feel like a modern hosting SaaS platform:

- Clear hero
- Fast domain search
- Strong pricing cards
- Simple checkout
- Professional dashboard
- Trustworthy billing
- Safe error handling
- Accessible for all users

The most important frontend rule is:

```
Make the customer journey simple, but keep all sensitive logic and
integrations secure on the backend.
```